

FICHE DE RÉVISION - Sécurité des Applications Mobiles

SÉANCE 1 : Comparaison Android vs iOS

1. Architectures de Sécurité

Critère	Android	iOS
Distribution	Open source, stores alternatifs possibles	App Store exclusif (revue stricte Apple)
Sandboxing	SELinux + App Sandbox (UID unique par app)	App Sandbox isolé + Code Signing obligatoire
Permissions	Runtime permissions (depuis Android 6.0)	Privacy prompts contextuels (depuis iOS 14)
Fragmentation	Élevée : 1000+ versions actives	Faible : >80% sur iOS 16+ en 6 mois
Mises à jour	Lentes (dépend fabricant/opérateur)	Rapides et uniformes (OTA Apple direct)
Stores tiers	Sideload APK possible (risque élevé)	Limité (EU DMA 2024) mais très restreint

2. Sécurité Offensive - Surface d'Attaque

Android

- **Surface d'attaque large** : sideloading APK, multiplicité OEM, fragmentation patches, root accessible
- **Attaques fréquentes** :
 - Malware trojanisé (app légitime modifiée)
 - Banking trojans (vol credentials bancaires)
 - Overlay attacks (fausse IHM superposée)
 - Exploitation de versions non patchées
- **Modèle d'attaquant** : Cybercriminel opportuniste (très actif), malware de masse (fréquent), APT étatique, Exploit Zero-day
- **Coût** : Exploitation souvent moins coûteuse
- **Conclusion** : Surface d'attaque large + attaques de masse

iOS

- **Surface d'attaque restreinte** : Écosystème fermé, signature obligatoire, jailbreak complexe
- **Attaques fréquentes** :
 - Exploits kernel zero-day (vulnérabilité non corrigée)
 - Chaînes d'exploitation sophistiquées (Pegasus-like)
 - Attaques via WebKit (moteur de rendu web)
- **Modèle d'attaquant** : Cybercriminel opportuniste (limité), malware de masse (rare), APT étatique, Exploit Zero-day (plus coûteux)

- **Coût** : Exploits rares, chers, sophistiqués
- **Conclusion** : Surface restreinte + attaques ciblées de haute sophistication

3. Sécurité Kernel / UserLand / Datas

Couche	Android	iOS
Kernel	Basé sur Linux, SELinux enforcing mode, Verified Boot	XNU (hybride Mach + BSD), Mandatory Code Signing, KASLR + Pointer Authentication
Environment	TEE (ARM TrustZone), isolation matérielle, exécution code sensible	Secure Enclave : coprocesseur dédié, mémoire isolée, génération/stockage clés crypto
Datas	Android Keystore (clés dans TEE), Support StrongBox	Data Protect : clés liées au hardware UID, protection par classe

Conclusions : - Android : dépend du constructeur pour la qualité du TEE - iOS : intégration matérielle plus verticale

4. Diagramme Architectural (niveau Kernel)

Android (3 couches)

1. **Couche UserLand** : Applications (UID unique/Sandbox), Android Framework/Runtime (ART), System Services (Binder IPC)
2. **Couche Kernel** : Linux Kernel, Process Isolation, SELinux MAC, Drivers
3. **Couche Matérielle** : Hardware, SoC, ARM TrustZone (TEE), Secure Element/StrongBox

iOS (3 couches)

1. **Couche UserLand** : Applications (Sandbox + Code Signing), Cocoa Touch/iOS Frameworks, Core Services/Security Services
2. **Couche Kernel** : XNU Kernel, Mandatory Code Signing, KASLR, Pointer Authentication
3. **Couche Matérielle** : Secure Enclave Processor (SEP), Clés cryptographiques, Biométrie (FaceID/TouchID), Hardware Apple Silicon

5. Modèles de Sécurité Mobile

Android

- **Linux Kernel** : Base de sécurité, isolation processus, permissions filesystem
- **HAL + ART** : Hardware Abstraction Layer, Android Runtime (AOT)
- **SELinux Enforcing** : Contrôle d'accès obligatoire (MAC) pour chaque processus
- **Verified Boot** : Démarrage vérifié cryptographiquement, intégrité système
- **SafetyNet / Play Integrity** : Attestation device, détection root, certification apps

iOS

- **Secure Boot Chain** : ROM → iBoot → Kernel → Apps, signature à chaque étape
 - **Kernel XNU (Darwin)** : Hybride Mach + BSD, séparation noyau/userspace stricte
 - **Secure Enclave (SEP)** : Cryptoprocresseur ARM isolé, clés biométriques, AES-256
 - **Code Signing** : Toute app signée par Apple Developer, validation à chaque lancement
 - **App Store Review** : Validation manuelle + automatisée, guidelines sécurité strictes
-

Paysage des Menaces Mobiles

Statistiques Clés

- **15%** des apps infectées (Malware Mobile)
- **30%** des apps vulnérables (Attaques Réseau)
- **+50%** de croissance/an (Phishing Mobile)
- **43%** des apps vulnérables (Vol de Données)

Types de Menaces

Malware Mobile

- Trojans : accès non autorisé, vol de données
- Spyware : surveillance, enregistrement
- Ransomware : chiffrement, rançon
- Banking Trojans : vol credentials bancaires

Attaques Réseau

- Man-in-the-Middle (MitM) sur HTTP
- Interception SSL / SSL Stripping
- Evil Twin : faux points d'accès WiFi
- DNS Spoofing et session hijacking

Phishing Mobile

- SMS Phishing (Smishing)
- Fausses applications (Fake Apps)
- QR Code malveillants (Quishing)
- Voice Phishing (Vishing)

Vol de Données

- Stockage non chiffré (SQLite, prefs)
- Logs et fichiers temporaires exposés
- Clipboard non sécurisé
- SD card accessible sans chiffrement

Vecteurs d'Attaque Spécifiques au Mobile

- **Apps Tierces Malveillantes** : Stores alternatifs, sideloading APK, >90% des infections mobiles
- **Permissions Excessives** : Caméra, micro, GPS, contacts accordés sans lecture (90%+)

- **Réseaux WiFi Publics** : Evil Twin, MitM, SSL Stripping (aéroports, cafés, hôtels)
- **Bluetooth & NFC** : BlueBorne RCE, Relay Attacks NFC, Jacking et écoute passive
- **Composants Exportés** : Activities/Services sans protection, Content Providers mal configurés

SÉANCE 3 : Architecture Android en Détail

Architecture Android en Couches

1. **Applications** : Gmail, Chrome, apps tierces — couche utilisateur
2. **Application Framework** : Activity Manager, Package Manager, APIs Java/Kotlin
3. **ART + Native Libraries** : Android Runtime (AOT), OpenSSL, SQLite, WebKit, libc
4. **HAL — Hardware Abstraction Layer** : Interface matériel standardisée (caméra, BT, GPS)
5. **Linux Kernel** : Drivers, gestion mémoire, processus, SELinux, namespaces

Le Sandbox Android — Isolation des Applications

Principe : Chaque app = un utilisateur Linux distinct

- **UID Linux unique par app** : Range 10000-19999, isolation totale des processus et fichiers
- **Répertoire privé /data/data/** : Permission 700 Unix, seule l'app peut accéder à ses données
- **Communication via IPC sécurisé** : Binder IPC, Intent avec permissions, ContentProvider avec URI grants
- **SELinux en mode Enforcing** : MAC (Mandatory Access Control), politiques strictes pour chaque domaine

Système de Permissions Android

4 Types de Permissions

Type	Description	Exemples
Normal	Accordées automatiquement sans confirmation	INTERNET, BLUETOOTH, RECEIVE_BOOT_COMPLETED
Dangerous	Nécessitent consentement explicite de l'utilisateur	CAMERA, LOCATION, CONTACTS, MICROPHONE, SMS, STORAGE
Signature	Réservées aux apps avec même certificat que le système	INSTALL_PACKAGES, STATUS_BAR, SET_ANIMATION_SCALE
Special	Action utilisateur spécifique dans les paramètres	SYSTEM_ALERT_WINDOW (overlay), WRITE_SETTINGS, MANAGE_OVERLAY

Évolution des Permissions

- **Android <6.0** : install-time (toutes demandées à l'installation)
- **Android 6.0+** : runtime permissions (demandées à l'utilisation)
- **Android 11+** : one-time + auto-reset

Permissions Dangereuses — 9 Groupes

Groupe	Permissions
CALENDAR	READ/WRITE_CALENDAR
CAMERA	CAMERA
CONTACTS	READ/WRITE/GET_ACCOUNTS
LOCATION	FINE/COARSE/BACKGROUND
MICROPHONE	RECORD_AUDIO
PHONE	READ_PHONE_STATE, CALL_PHONE, READ_CALL_LOG
SENSORS	BODY_SENSORS
SMS	SEND/RECEIVE/READ_SMS
STORAGE	READ/WRITE_EXTERNAL_STORAGE

Runtime Permissions — Flux d'Implémentation

Kotlin : `checkSelfPermission` → `requestPermissions` → `onRequestPermissionsResult`

Flux de demande

1. `checkSelfPermission()` : vérifier si permission accordée
2. `shouldShowRequestPermissionRationale()` : expliquer pourquoi
3. `requestPermissions(arrayOf(Manifest.permission.CAMERA), 100)`
4. `override onRequestPermissionsResult()` : gérer résultat

Bonnes pratiques

- Demander au moment de l'utilisation (contextuel)
- Expliquer clairement l'utilité à l'utilisateur
- Gérer gracieusement le refus (dégradation fonctionnelle)
- Android 11+ : gérer le 'Don't ask again'

Pièges fréquents

- Demander toutes les permissions au lancement (bad UX)
- Ne pas vérifier à chaque utilisation (permission révocable)
- Forcer l'utilisateur sans explication
- Crasher si permission refusée

Composants Android et Sécurité

4 Types de Composants

- **Activity** : Interface utilisateur (`exported=false` par défaut si pas d'intent-filter)
- **Service** : Traitement en arrière-plan (background)
- **BroadcastReceiver** : Écoute événements système (BOOT, SMS...)
- **ContentProvider** : Partage de données entre applications

Risque : `exported=true` sans protection

- Activity exportée → n'importe quelle app peut la lancer
- Service exporté → exécution de code arbitraire
- ContentProvider sans permission → accès aux données

- BroadcastReceiver → injection d'intents malveillants

Protection recommandée

- android:exported="false" si pas d'accès inter-apps nécessaire
- android:permission pour restreindre l'accès
- Valider les intents reçus (Intent validation)
- Signature-level protection pour composants sensibles

Android KeyStore — Stockage Sécurisé des Clés

Principe : Clés cryptographiques non extractibles, protégées par TEE/StrongBox

Caractéristiques KeyStore

- Clés non extractibles (restent dans le hardware)
- Protection par TEE (ARM TrustZone) ou StrongBox
- Génération : RSA, AES, HMAC, EC
- Authentification requise (biométrie, PIN) optionnelle

Opérations disponibles

- Chiffrement/déchiffrement (AES-GCM, RSA-OAEP)
- Signature/vérification (ECDSA, RSA)
- Génération de clés directement dans le hardware
- Import de clés externes (avec précautions)

Architecture

- App → KeyStore API → keystore daemon → TEE/StrongBox
- KeyStore API : KeyPairGenerator, KeyGenerator, Cipher
- Attributs : userAuthenticationRequired, setKeyValidityEnd...
- StrongBox (Android 9+) : HSM dédié, résistance physique

TEE — Trusted Execution Environment

Définition : Zone d'exécution sécurisée isolée du système principal par le hardware

ARM TrustZone

- Processeur divisé : Normal World (Android) vs Secure World
- Isolation hardware totale entre les deux mondes
- Changement de mode via SMC (Secure Monitor Call)
- Exécution du code TEE en Secure World isolé

StrongBox (Android 9+)

- Hardware Security Module (HSM) complètement séparé
- CPU, mémoire, stockage dédiés et isolés
- Résistance aux attaques physiques (side-channel)
- Capacité limitée (pas de RSA-4096) mais maximale sécurité

Usages dans Android

- Android KeyStore : stockage clés cryptographiques
- Biométrie : templates d'empreintes/face dans TEE
- WideVine DRM : protection de contenu premium
- Trusted UI : écrans de saisie PIN isolés

EncryptedSharedPreferences et Stockage Sécurisé

Jetpack Security Library — chiffrement transparent et automatique

EncryptedSharedPreferences

- Chiffrement AES-256-GCM pour les valeurs
- AES-256-SIV pour les clés (déterministe)
- Utilise Android KeyStore automatiquement
- Drop-in replacement pour SharedPreferences

EncryptedFile

- Chiffrement de fichiers via AES-256-GCM
- Lecture/écriture transparente avec streams Java
- Protection contre extraction des fichiers
- Compatible avec Room Database chiffrée

SQLCipher pour Room

- Chiffrement AES-256 de toute la base SQLite
- Clé stockée dans Android KeyStore
- Compatible avec Room ORM (bibliothèque SQLCipher)
- Performance légèrement réduite (~10%)

SÉANCE 5 : Architecture iOS en Détail

Architecture iOS — Vue d'Ensemble

Système fermé et propriétaire : intégration hardware-software optimale

Stack iOS (du bas vers le haut)

1. **Core OS** : Kernel XNU (Mach + BSD), sécurité bas niveau
2. **Core Services** : Foundation, Core Data, CloudKit
3. **Media Layer** : AVFoundation, Core Graphics, Metal
4. **Cocoa Touch** : UIKit, SwiftUI, ARKit, frameworks UI

Caractéristiques de sécurité iOS

- Intégration hardware-software (Apple Silicon / A-series)
- Mises à jour OTA rapides et uniformes (>85% en 6 mois)
- Compatibilité limitée mais contrôlée (moins de fragmentation)
- Jailbreak = perte quasi-totale des protections de sécurité

Secure Boot Chain — Chaîne de Confiance

Architecture ARM séparée, mémoire isolée, OS propre (sepOS)

Architecture Secure Enclave

- Coprocesseur ARM dédié, séparé du Application Processor
- Système d'exploitation propre : sepOS (microkernel Mach)
- Mémoire chiffrée isolée, inaccessible au kernel principal
- Boot sécurisé indépendant de la chaîne iOS principale

Fonctions du Secure Enclave

- Stockage et gestion des clés cryptographiques (non exportables)
- Traitement biométrique : Touch ID / Face ID (templates)
- Apple Pay : Secure Element pour paiements NFC
- True Random Number Generator (TRNG) hardware
- Chiffrement AES-256 des données utilisateur

Étapes du Secure Boot Chaque composant est vérifié par le précédent avant exécution

1. **Boot ROM (iBoot)** : code immuable en ROM, racine de confiance
2. **LLB (Low-Level Bootloader)** : vérifié par signature Apple
3. **iBoot (2nd stage)** : vérifié par LLB
4. **iOS Kernel (XNU)** : vérifié par iBoot
5. **Kernel Extensions** : vérifiées par le kernel

Propriétés de sécurité

- Signature cryptographique Apple à CHAQUE étape
- Échec de vérification = arrêt immédiat (mode recovery/DFU)
- Impossible de booter un OS non signé par Apple
- Protection contre rollback via nonces cryptographiques (SHSH)

Code Signing — Signature Obligatoire

Toute application doit être signée avec un certificat Apple Developer valide

Types de certificats développeur

- **Development** : tests sur devices enregistrés (<100)
- **Ad Hoc** : distribution limitée à 100 devices maximum
- **App Store** : distribution publique via App Store
- **Enterprise** : distribution interne entreprise (In-House)

Profil de provisioning (embedded.mobileprovision)

- Lie : certificat développeur + App ID (bundle ID) + Devices
- Contient les entitlements autorisés pour l'application
- Signé par Apple, embarqué dans l'IPA au build
- Expiration : 7 jours (dev gratuit) ou 1 an (compte payant)

Vérification au lancement

- Signature vérifiée à chaque lancement de l'app
- Validation de l'intégrité du binaire (tampering detection)
- Vérification des entitlements contre profil provisioning

App Store Review — Processus de Validation

Barrière de sécurité : double validation automatique + manuelle par Apple

Étapes du processus de review

1. Analyse automatisée : malware scan, API privées, frameworks
2. Review manuelle : conformité guidelines, contenu, fonctionnalités
3. Tests fonctionnels : crash, comportement anormal
4. Vérification métadonnées : screenshots, descriptions, privacy policy

Critères de rejet courants

- Utilisation d'API privées (Private APIs)
- Collecte de données sans consentement / sans purpose
- Contenu inapproprié ou trompeur
- Bugs critiques, crashes au lancement
- Guideline 4.3 : spam / fonctionnalité dupliquée

Limites de la review

- Non infaillible (malware détecté après publication)
- Délai : 24-48h en général, jusqu'à plusieurs semaines
- Bypass possibles via code distant (scripts JS hot-patching)

iOS Sandbox et Permissions

Container isolé par app + permissions runtime granulaires

Structure du container d'une app iOS

- **Bundle/** : Binaire et ressources (lecture seule, signé)
- **Documents/** : Données utilisateur (sauvegardées iCloud/iTunes)
- **Library/** : Caches, Application Support, Preferences
- **tmp/** : Fichiers temporaires, non sauvegardés

Principales permissions iOS (Info.plist)

- NSCameraUsageDescription (obligatoire depuis iOS 10)
- NSLocationWhenInUseUsageDescription
- NSMicrophoneUsageDescription
- NSContactsUsageDescription, NSPhotoLibraryUsageDescription
- NSFaceIDUsageDescription (Face ID)

Keychain et Data Protection iOS

Stockage sécurisé matériel + classes de protection par fichier

Keychain Services

- Base de données chiffrée pour données sensibles (AES-256)
- Géré par securityd, clés dans Secure Enclave
- Types : GenericPassword, InternetPassword, Certificate, Key, Identity
- Accessibilité : `kSecAttrAccessibleWhenUnlocked` (recommandé)
- Synchronisation iCloud optionnelle (`kSecAttrSynchronizable`)

Data Protection — 4 classes

Classe	Description
NSFileProtectionComplete	Inaccessible device verrouillé (max sécurité)
NSFileProtectionCompleteUnlessOpen	Ouvert avant verrouillage
NSFileProtectionCompleteUntilFirstUserAuthentication	Verrouillage à l'authentification (défaut)
NSFileProtectionNone	Toujours accessible (à éviter pour données sensibles)

POINTS CLÉS À RETENIR

Android

1. **Sandbox** basé sur UID Linux unique + SELinux
2. **4 types de permissions** : Normal, Dangerous, Signature, Special
3. **Runtime permissions** depuis Android 6.0
4. **9 groupes de permissions dangereuses**
5. **4 composants** : Activity, Service, BroadcastReceiver, ContentProvider
6. **KeyStore** avec TEE/StrongBox pour clés cryptographiques
7. **EncryptedSharedPreferences** pour stockage sécurisé

iOS

1. **Secure Boot Chain** : vérification à chaque étape
2. **Code Signing obligatoire** avec certificat Apple
3. **Secure Enclave** : coprocesseur dédié sécurité
4. **App Store Review** : validation automatique + manuelle
5. **Keychain** pour données sensibles
6. **Data Protection** : 4 classes de protection fichiers
7. **Sandbox** avec container isolé par app

Comparaison Clé

- **Android** : Plus ouvert, plus de surface d'attaque, attaques de masse
- **iOS** : Plus fermé, surface restreinte, attaques ciblées sophistiquées